

UNIVERSIDAD DE GRANADA

Arquitecturas y Computación de Altas Prestaciones

Practica 2 Parte 2

Ignacio Palomares Martín

1. Introducción y Objetivo

En esta práctica he implementado el cálculo de inferencia de una capa de red neuronal densa. En el fondo, esto no deja de ser una multiplicación de matrices masiva seguida de una función de activación (en este caso, he usado ReLU). La fórmula matemática que define la matriz de salida **Y** a partir de los datos de entrada **X** y los pesos **W** de la red

$$\mathbf{Y = ReLU(X * W)}$$

Como el cálculo de cada valor de la matriz resultado es totalmente independiente del resto, es un problema perfecto para paralelizarlo. Usando CUDA, he vectorizado la ejecución asignando un hilo de la tarjeta gráfica a cada coordenada de la matriz final, acelerando el proceso enormemente en comparación con una CPU tradicional.

2. Diseño de la Solución

Para desarrollar el código en C++, he programado dos *kernels* distintos. Hacerlo así me ha servido para comprobar empíricamente cómo afecta el acceso a memoria al rendimiento real de la tarjeta.

2.1. Kernel Directo (Memoria Global)

La primera versión es la más intuitiva (pero la más lenta). Cada hilo lee la fila y la columna que necesita directamente de la memoria de vídeo principal (VRAM) y acumula las sumas. El problema de este enfoque es que genera millones de accesos repetidos a los mismos datos, saturando el ancho de banda del bus. Esto provoca que los núcleos matemáticos de la gráfica se queden parados esperando a que lleguen los datos.

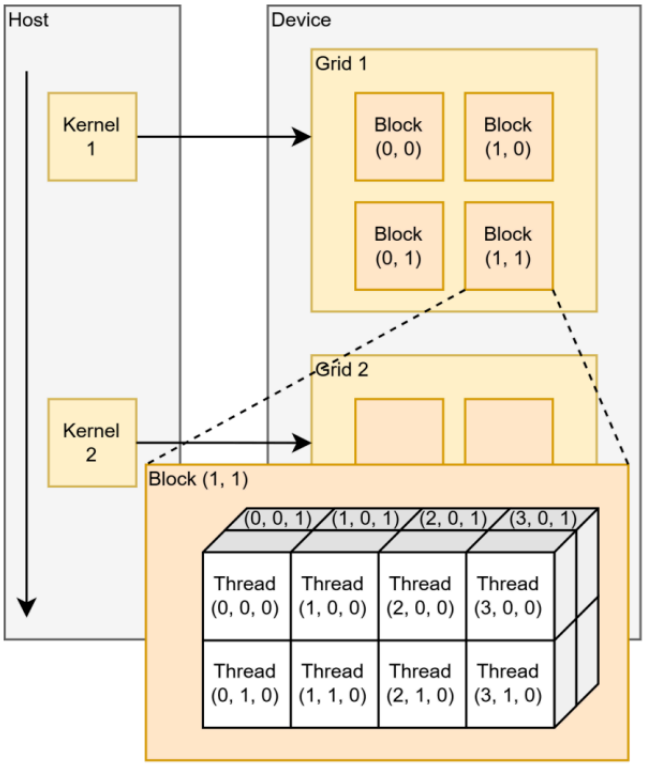
2.2. Kernel Optimizado (Tiling y Memoria Compartida)

Para solucionar este cuello de botella, he implementado la técnica de *Tiling* (mosaicos). He configurado los bloques de hebras para que trabajen en equipo: cargan de forma colaborativa un "azulejo" de datos de la memoria global hacia

la memoria compartida (__shared__). Como esta memoria está físicamente dentro del propio multiprocesador, la latencia es mínima.

Una vez que el azulejo entero está cargado, sincronizo las hebras con __syncthreads() para asegurar que todas tienen los datos listos, hacen los cálculos a máxima velocidad, y pasan al siguiente azulejo.

```
1  __syncthreads();
2
3  for (int k = 0; k < TILE_WIDTH; ++k) {
4      value += ds_X[ty][k] * ds_W[k][tx];
5   }
6
7  __syncthreads();
```



3. Entorno de Pruebas

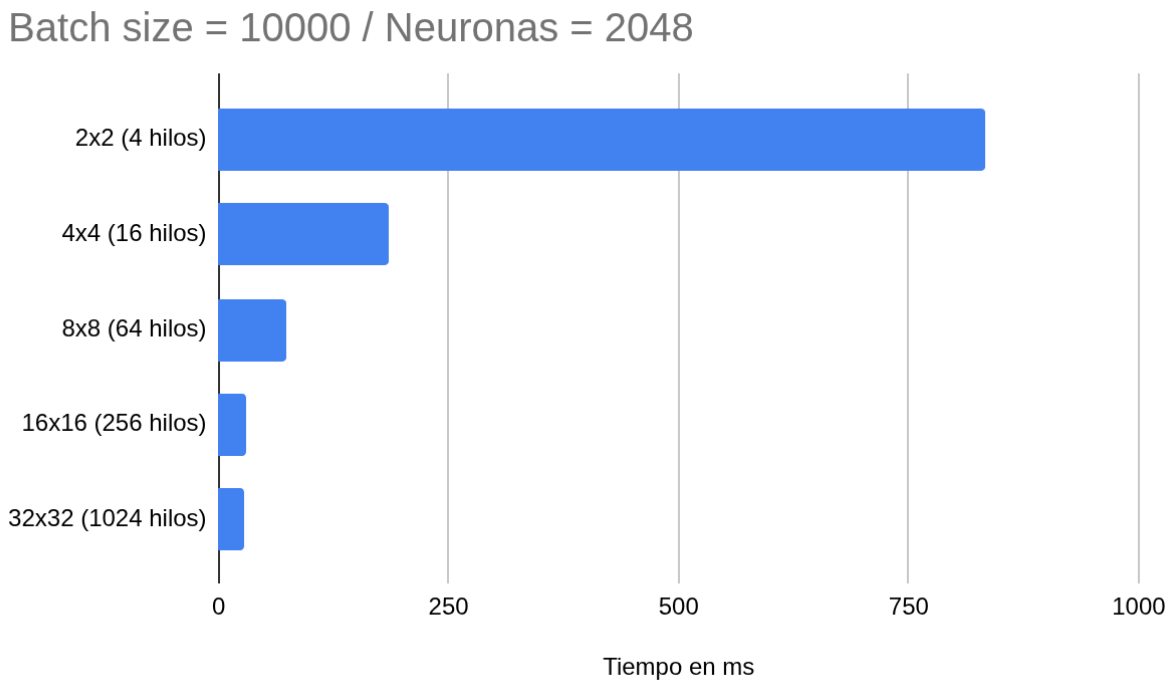
Las pruebas se han ejecutado en un nodo equipado con un procesador anfitrión (Host) **Intel Core i7-9700F a 3.00 GHz**, acompañado de una tarjeta gráfica **NVIDIA RTX 2080 Ti** (11 GB VRAM). El código fuente lo he compilado con ***nvcc*** usando el flag de máxima optimización (***-O3***).

Para medir los tiempos de ejecución en milisegundos de forma precisa, y evitar que el tiempo de copia de datos entre la placa base y la gráfica falseara los resultados, he usado la API de eventos nativa de CUDA (***cudaEvent_t***).

4. Resultados y Análisis de Rendimiento

4.1. Ocupación del Multiprocesador

TABLA 1: CURVA DE OCUPACIÓN DEL MULTIPROCESADOR				
Prueba	Hilos x Bloque (X*Y)	Batch Size	Neuronas	Tiempo CUDA (ms)
Extremo inferior	2x2 (4 hilos)	10000	2048	833,289
Bajo	4x4 (16 hilos)	10000	2048	184,004
Sub-óptimo	8x8 (64 hilos)	10000	2048	74,3387
Óptimo Teórico	16x16 (256 hilos)	10000	2048	29,2474
Límite hardware	32x32 (1024 hilos)	10000	2048	27,1567

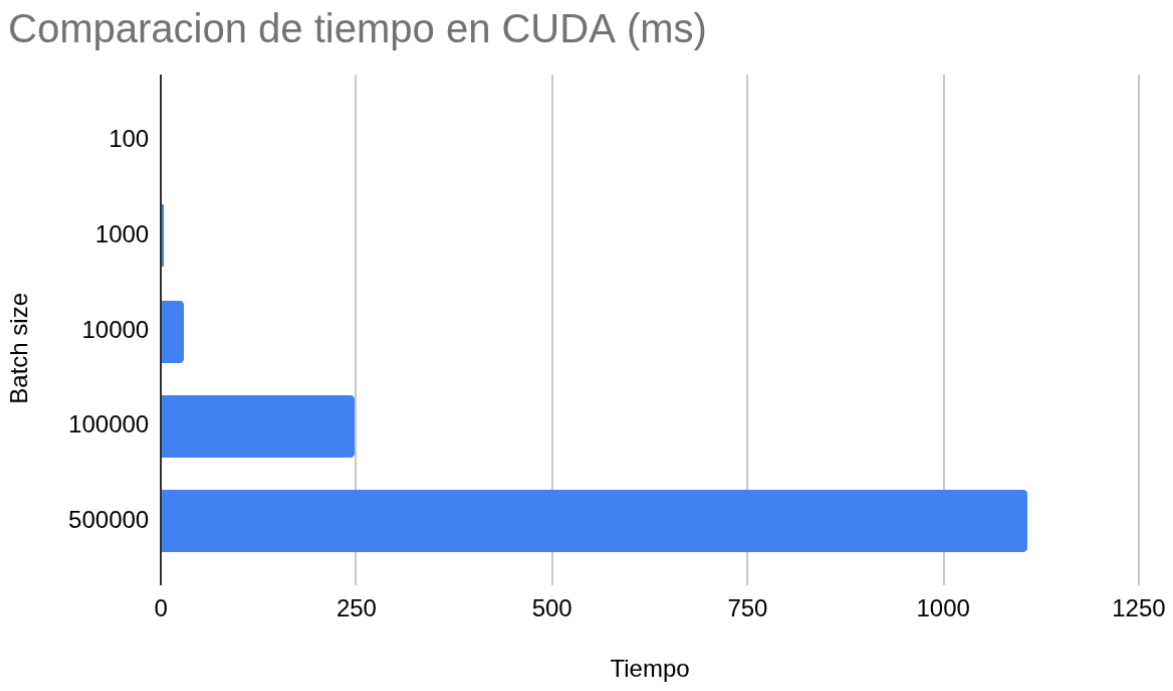


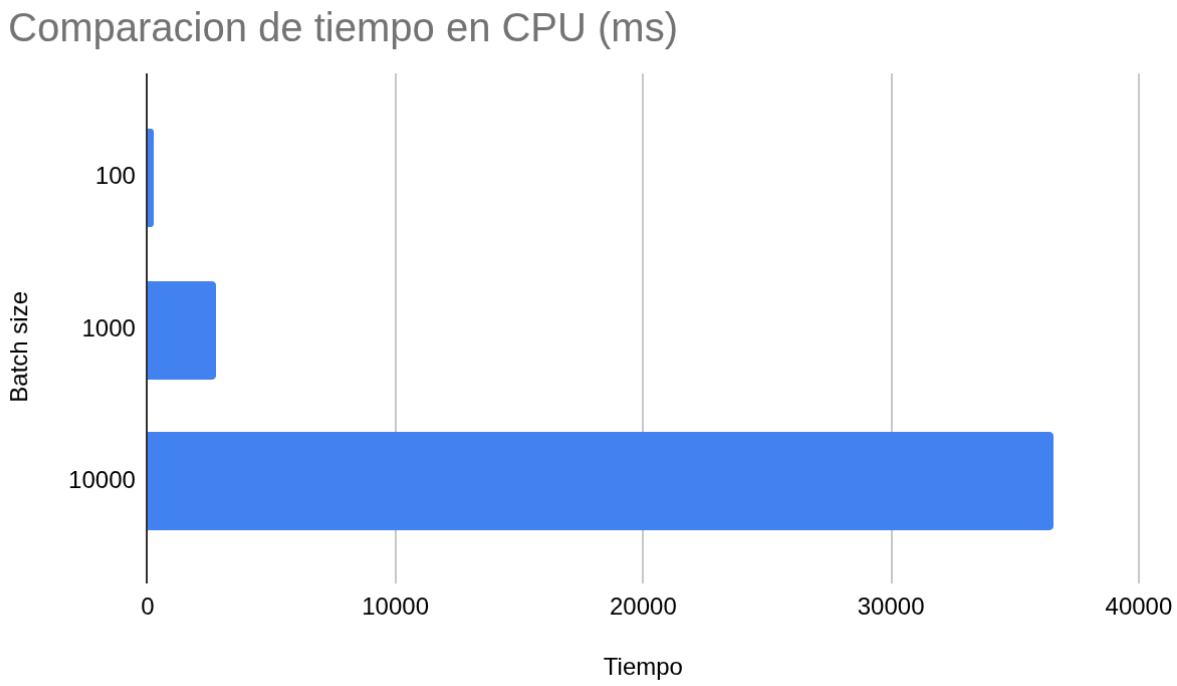
En esta prueba mantuve la carga de trabajo fija (10.000 datos y 2.048 neuronas) y fui cambiando el tamaño de la topología del bloque en el código. Como se ve en los tiempos, lanzar bloques muy pequeños (como 2x2 o 4x4) penaliza mucho el rendimiento. Al haber tan pocos hilos, desaprovechamos los recursos del multiprocesador porque el planificador no tiene suficientes *warps* activos para ir turnándolos y ocultar los retrasos de memoria.

El punto óptimo lo he encontrado con bloques de 16x16 (256 hebras), que me ha dado un tiempo de 29,24 ms con un equilibrio muy eficiente de recursos. El bloque de 32x32 fue el más rápido por muy poco (27,15 ms) al saturar el límite máximo de hardware de 1024 hebras por bloque que permite la gráfica.

4.2. Escalabilidad de Datos (Batch Size)

TABLA 2: ESCALABILIDAD DE DATOS - BATCH SIZE				
Batch Size (Nº de Datos)	Características	Neuronas	Tiempo CUDA (ms)	Tiempo CPU (ms)
100	1024	2048	0,392096	276,021
1000	1024	2048	2,99821	2742,91
10000	1024	2048	29,2439	36520,4
100000	1024	2048	248,274	Omitido (Tardaría demasiado)
500000	1024	2048	1106,67	Omitido (Tardaría demasiado)
1000000	1024	2048	**Límite físico superado (Demanda de 12.28 GB frente a 11 GB disponibles).**	





Aquí comparé el tiempo de ejecución secuencial en la CPU frente a la GPU aumentando la cantidad de datos progresivamente. La CPU se ahoga enseguida (tardó más de 36 segundos para procesar solo 10.000 datos). En cambio, la GPU escala excepcionalmente bien. Al inyectarle más datos, el hardware amortiza el coste estructural porque tiene miles de hilos en vuelo que le permiten enmascarar las latencias (*Latency Hiding*).

Además, llevé la gráfica a su límite físico: al pedirle procesar un lote masivo de 1.000.000 de datos, intentó reservar un total de 12,28 GB en memoria. El programa lo detectó y abortó de forma controlada lanzando un error de memoria (OOM), ya que superaba los 11 GB físicos de la RTX 2080 Ti .

4.3. Profundidad Computacional y Anchura de Red

TABLA 3: ANCHURA DE LA RED - NÚMERO DE NEURONAS			
Batch Size	Características	Neuronas (Ancho capa)	Tiempo CUDA (ms)
10000	1024	256	3,70147
10000	1024	512	7,34010
10000	1024	1024	14,6136
10000	1024	2048	29,2559
10000	1024	4096	58,387
10000	1024	8192	118,266

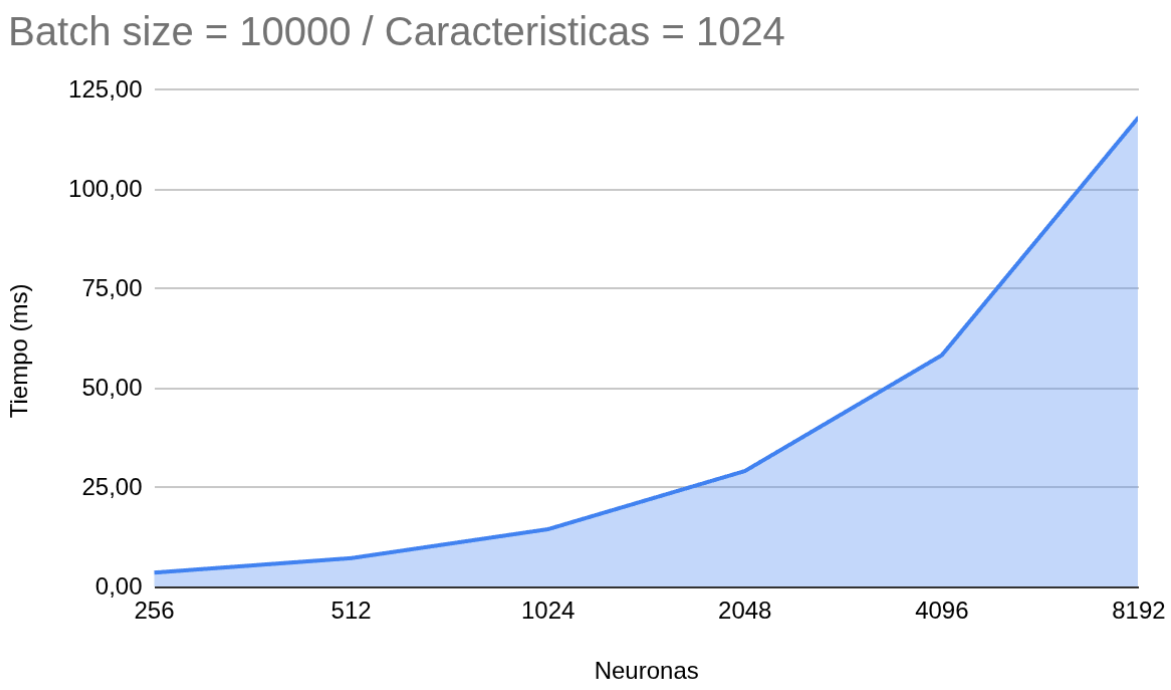
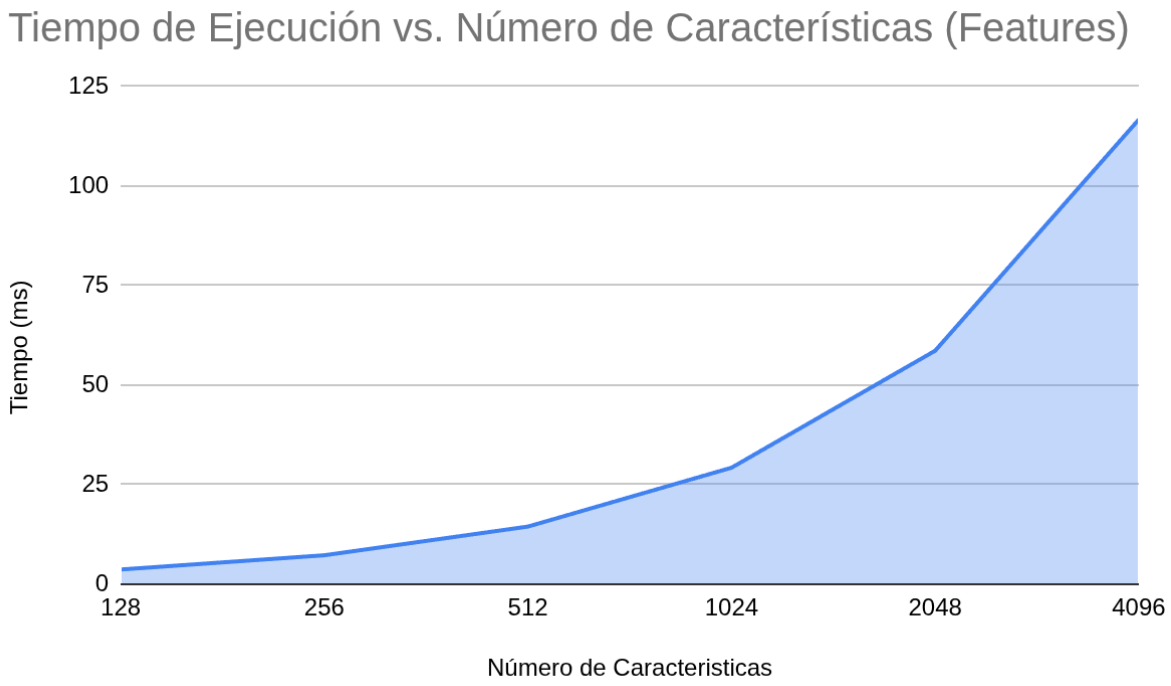


TABLA 5				
Batch Size	Neuronas	Características (Features)	Ciclos del Bucle (Tiles)	Tiempo Compartida 16x16 (ms)
10000	2048	128	8	3,68374
10000	2048	256	16	7,25946
10000	2048	512	32	14,4651
10000	2048	1024	64	29,2537
10000	2048	2048	128	58,6275
10000	2048	4096	256	116,679

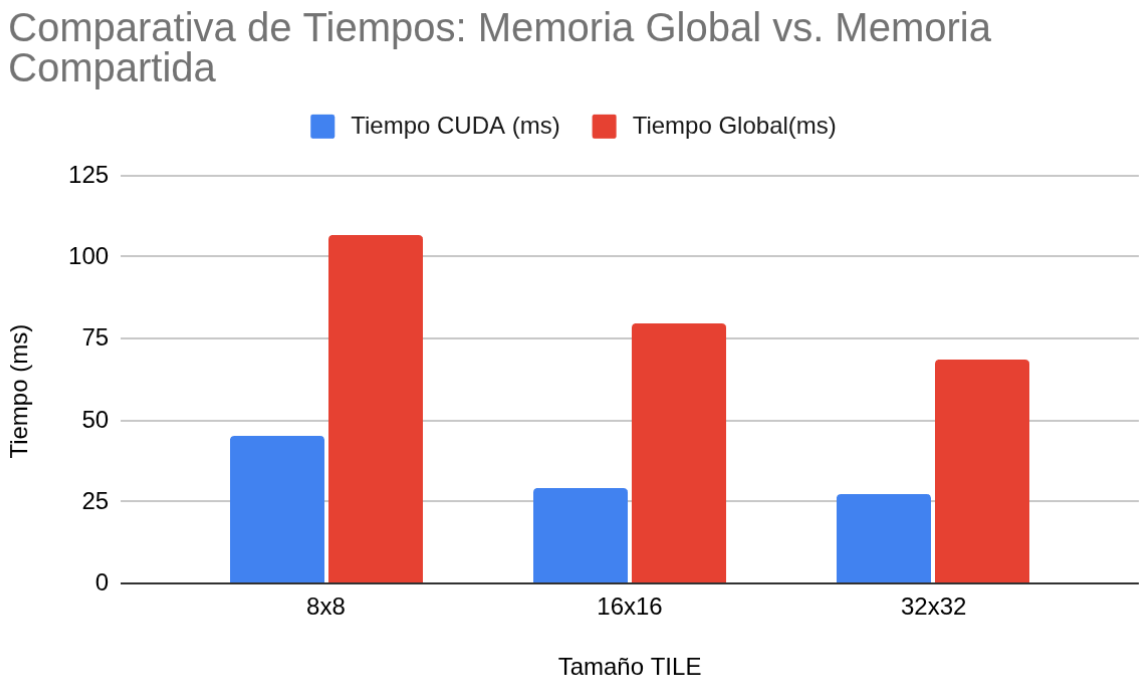


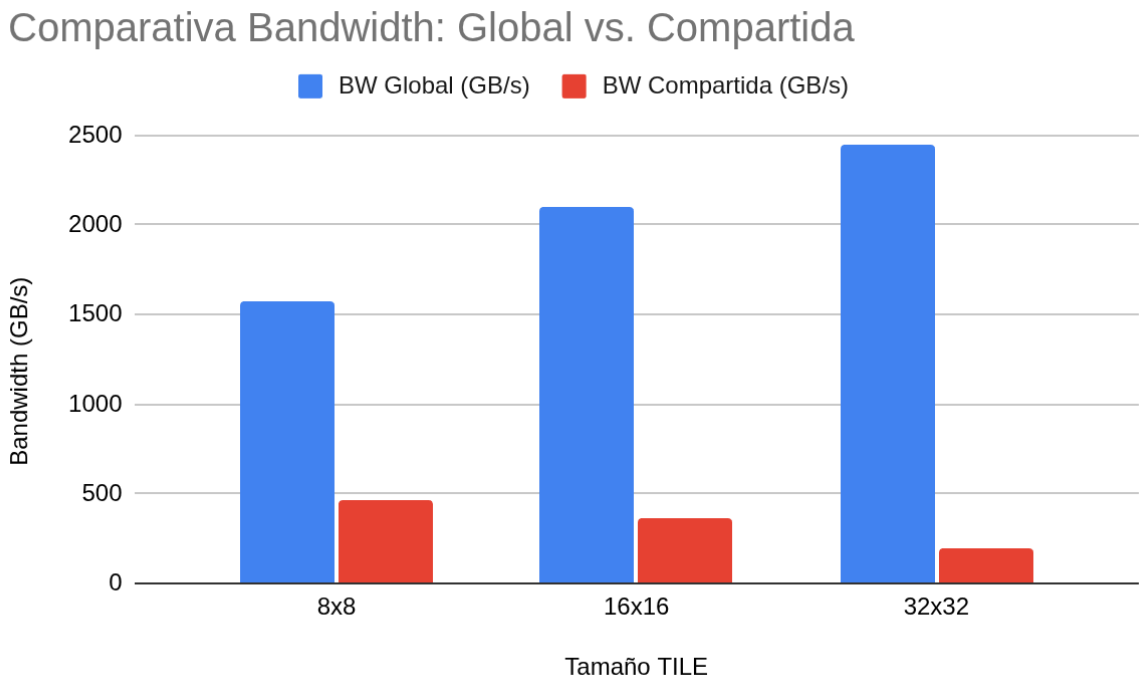
Con esta batería de pruebas quería ver qué pasaba si aumentaba mucho la carga matemática pura (multiplicaciones y sumas de las ALUs) sin aumentar en exceso la memoria demandada. Al subir el número de características (hasta 4096) y neuronas, el tiempo sube de forma bastante lineal y predecible.

Esto demuestra que el kernel optimizado es muy robusto: al mantener temporalmente los datos en la memoria __shared__, la tarjeta es capaz de tragarse todo ese incremento bestial de multiplicaciones sin que el bus de memoria global colapse.

4.4. Eficiencia de Memoria Compartida (Tiling) frente a Global

TABLA 4: OPTIMIZACIÓN DE MEMORIA Y TILING							
Batch Size	Neuronas	Tamaño TILE (Mosaico)	Tiempo CUDA (ms)	Tiempo Global(ms)	Speedup	BW Global (GB/s)	BW Compartida (GB/s)
10000	2048	8x8	44,9188	106,824	2,37816	1572,025013	468,8460066
10000	2048	16x16	29,0303	79,8845	2,75177	2102,159993	364,1023345
10000	2048	32x32	27,1379	68,5534	2,52612	2449,623213	196,0358023





Esta es la prueba que justifica la práctica. Calculando el ancho de banda efectivo exigido, descubrí que el kernel sin optimizar (Global) le pide a la gráfica mover más de 2.000 GB/s. Esto es físicamente imposible (el límite teórico de la 2080 Ti es de unos 616 GB/s), lo que asfixia a la gráfica y explica el mal tiempo de 79,88 ms.

Para cuantificar la eficiencia de la transferencia de datos, he calculado el ancho de banda efectivo empleando la siguiente relación matemática:

BW_efectivo = (Bytes_Leídos + Bytes_Escritos) / (Tiempo * 10^9) GB/s

Al usar *Tiling* con bloques de 16x16, el tráfico de datos cruzados hacia la memoria principal cae en picado. El ancho de banda efectivo se queda en unos manejables 364,1 GB/s y el tiempo bajó a 29,03 ms. En resumen, he conseguido un *Speedup* (aceleración pura) de **2,75x** respecto a la versión inicial.

5. Conclusiones

Desarrollar esta práctica me ha dejado claro que, en el ámbito de la Computación de Altas Prestaciones, el verdadero cuello de botella casi nunca es la falta de potencia matemática de los procesadores, sino el coste de mover los

datos. De poco sirve tener miles de núcleos capaces de sumar muy rápido si se pasan la mayor parte del tiempo inactivos esperando a la VRAM.

Aprovechar correctamente la jerarquía de memoria de la arquitectura CUDA, organizando el trabajo en mosaicos y usando la memoria compartida local, es un paso obligatorio si queremos exprimir de verdad el hardware, evitar saturar el bus de datos y conseguir buenos tiempos de ejecución.